INFORME DE LABORATORIO #2

Expresiones Regulares con JFLAP

Teoría de la Computación – INFO1148

Antecedente	Información
Asignatura	Teoría de la Computación INFO1148
Carrera	Ingeniería Civil en Informática
Docente	M. Lévano
Integrantes	Francisco Benavides Affeld – Marcelo Matamala Núñez
Fecha	10 de octubre de 2026

1. Introducción

El propósito de este laboratorio es aplicar los conceptos de lenguajes regulares y expresiones regulares mediante la herramienta JFLAP, relacionando una expresión regular con los autómatas finitos que reconocen el mismo lenguaje. Para ello se interpreta una expresión regular dada, se diseñan expresiones propias a partir de restricciones sobre cadenas y se realizan las conversiones Expresión Regular $\rightarrow$ AFN $\rightarrow$ AFD, comprobando mediante casos de prueba la equivalencia entre estas tres representaciones de un lenguaje regular y justificando formalmente los resultados observados.

2. Desarrollo

2.1 Pregunta 1 – Interpretación y validación de una expresión regular

a) Lenguaje representado

La expresión regular `(0+1)*01` se compone de dos partes: **(0+1)***, que representa cualquier cadena de ceros y unos (incluida la cadena vacía), y **01**, que exige que la cadena termine con un **0** seguido de un **1**. Por lo tanto, la expresión representa el lenguaje de todas las cadenas sobre $\Sigma = \{0, 1\}$ que **terminan en "01"**:

$$L = \{w \in \{0,1\}^* \mid w \text{ termina en } 01\}$$

b) Resultados obtenidos

Tabla 1: Predicción y resultado en JFLAP para $(0+1)^*01$

Cadena	Predicción	Resultado JFLAP	¿Coinciden?
01	Aceptada	Aceptada	Sí
101	Aceptada	Aceptada	Sí
1101	Aceptada	Aceptada	Sí
0001	Aceptada	Aceptada	Sí
10	Rechazada	Rechazada	Sí
111	Rechazada	Rechazada	Sí
0100	Rechazada	Rechazada	Sí
10101	Aceptada	Aceptada	Sí

Predicciones y resultados de JFLAP coincidieron en las 8 cadenas, lo que valida la interpretación del lenguaje descrito por la expresión regular.

c) Evidencia en JFLAP

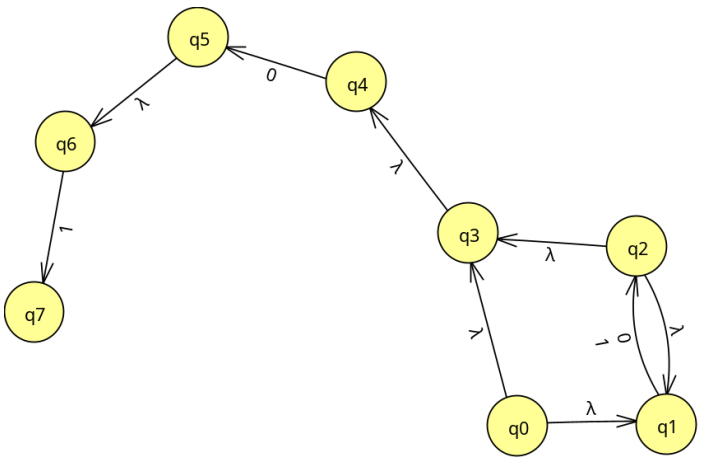


Figura 1: AFN obtenido en JFLAP para $(0+1)^*01$ (`P1_AFN.jff`)

d) Análisis

La cadena **1101 es aceptada** porque termina en "01": la parte $(0+1)^*$ absorbe el prefijo "11" y los dos últimos símbolos cumplen el sufijo 0 seguido de 1, por lo que el AFN finaliza en un estado de aceptación. La cadena **0100 es rechazada** porque termina en "00": aunque contiene la subcadena "01" al inicio, la expresión no exige que la cadena contenga "01", sino que *termine* en "01"; al final el símbolo leído es 0 y no 1, quedando el autómata en un estado no final.

2.2 Pregunta 2 – Diseño de una expresión regular

a) Expresión regular propuesta

- **Lenguaje:** $L_2 = \{w \in \{a, b\}^* \mid w$ comienza con a y termina con $b\}$
- **Expresión regular:** `a(a+b)*b`

b) Explicación

El **a** inicial obliga a que toda cadena válida comience con el símbolo **a**; **(a+b)*** permite cualquier secuencia intermedia de símbolos a y b, incluida la cadena vacía; y el **b** final obliga a que la cadena termine con el símbolo **b**. Se utiliza "+" para la unión por ser la sintaxis de JFLAP. La cadena intermedia vacía está permitida, por lo que la longitud mínima del lenguaje es **ab**.

c) Casos de prueba

Tabla 2: Casos de prueba para $a(a + b)^*b$

Cadena	Resultado esperado	Resultado JFLAP
ab	Aceptada	Aceptada
aab	Aceptada	Aceptada
abb	Aceptada	Aceptada
abab	Aceptada	Aceptada
aabab	Aceptada	Aceptada
aababab	Aceptada	Aceptada
a	Rechazada	Rechazada
b	Rechazada	Rechazada
ba	Rechazada	Rechazada
bb	Rechazada	Rechazada
aba	Rechazada	Rechazada
baba	Rechazada	Rechazada

Se probaron 6 cadenas aceptadas, 6 rechazadas, la de longitud mínima (`ab`) y dos casos límite (`a`, que cumple el inicio pero carece del b final; y `b`, que termina en b pero no empieza con a). Todos los resultados coincidieron con lo esperado.

d) Evidencia en JFLAP

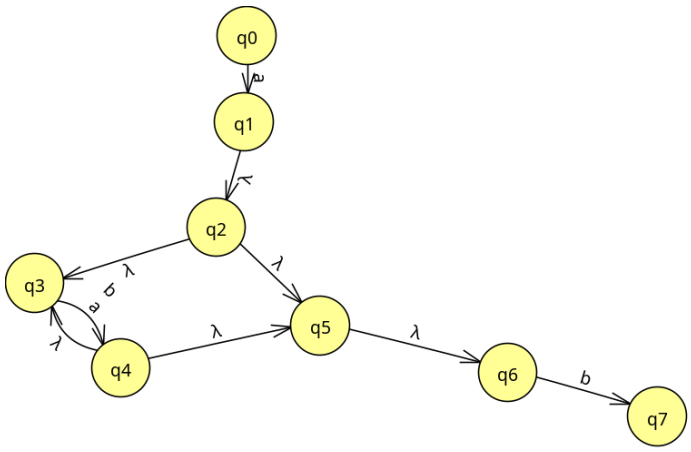


Figura 2: AFN generado para $a(a + b)^*b$ (`P2_AFN.jff`)

e) Análisis

La expresión regular satisface ambas restricciones de forma independiente: cada parte controla exactamente una condición (inicio, parte intermedia y final), y las pruebas confirman que las cadenas aceptadas son precisamente las que comienzan con a y terminan con b, incluyendo los casos límite de longitud mínima y de un único símbolo.

2.3 Pregunta 3 – Conversión ER $\rightarrow$ AFN $\rightarrow$ AFD

a) Expresión regular propuesta

- **Lenguaje:** $L_3 = \{w \in \{0,1\}^* \mid w \text{ contiene la subcadena } 101\}$
- **Expresión regular:** `(0+1)*101(0+1)*`

Los $(0+1)^*$ inicial y final permiten cualquier prefijo y sufijo, de modo que la subcadena "101" puede aparecer en cualquier posición de la cadena, describiendo exactamente el lenguaje de las cadenas que contienen la subcadena 101.

b) Características de los autómatas

Tabla 3: Comparación de características AFN vs AFD para L_3

Característica	AFN	AFD
Número de estados	14	4
Número de transiciones	19	8
Transiciones λ	Sí (12)	No (0)
Determinista	No	Sí

El AFN no es determinista porque posee transiciones λ y admite más de un camino para un mismo símbolo; el AFD, por el contrario, tiene exactamente una transición por estado y símbolo y no posee transiciones λ.

c) Pruebas realizadas

Tabla 4: Resultados de pruebas en AFN y AFD para subcadena 101

Cadena	Resultado AFN	Resultado AFD	¿Coinciden?
101	Aceptada	Aceptada	Sí
0101	Aceptada	Aceptada	Sí
1010	Aceptada	Aceptada	Sí
11011	Aceptada	Aceptada	Sí
000	Rechazada	Rechazada	Sí
111	Rechazada	Rechazada	Sí
1001	Rechazada	Rechazada	Sí
1101010	Aceptada	Aceptada	Sí

d) Evidencias

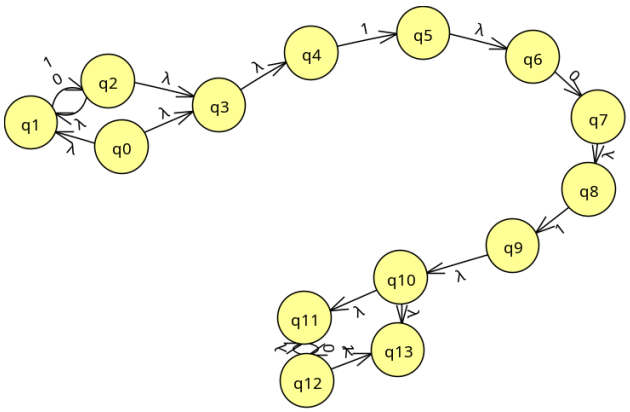


Figura 3: AFN generado (`P3_AFN.jff`)

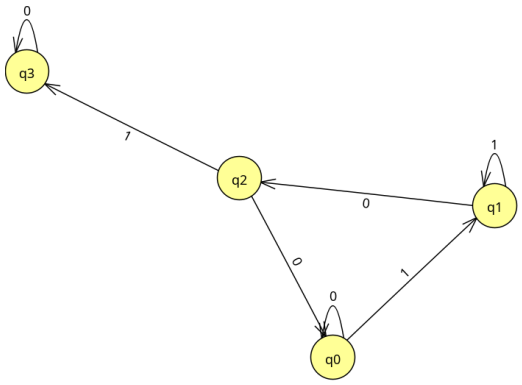


Figura 4: AFD equivalente (`P3_AFD.jff`)

e) Análisis

El AFN y el AFD **reconocen exactamente el mismo lenguaje**. El AFD se obtuvo a partir del AFN mediante la conversión por subconjuntos, que por construcción preserva el lenguaje: cada estado del AFD representa un conjunto de estados del AFN alcanzables al leer el mismo prefijo. Esto se confirma empíricamente: en las 8 cadenas probadas ambos autómatas entregaron resultados idénticos, y una verificación exhaustiva sobre todas las cadenas de longitud ≤ 7 no encontró ninguna cadena aceptada por uno y rechazada por el otro. Las diferencias entre ambos son de representación (no determinismo y transiciones λ frente a determinismo), no de poder de reconocimiento.

2.4 Pregunta 4 – Diseño y validación

a) Expresión regular propuesta

$$\text{ER} = \texttt{(a+b)*a(a+b)*bb}$$

b) Justificación

La parte **(a+b)*a(a+b)*** es la forma clásica de describir “contiene al menos una a”: el símbolo a aparece obligatoriamente en algún punto, con libre tránsito de a y b antes y después de él. El **bb** final obliga a que los dos últimos símbolos de la cadena sean b, b. Ambas condiciones no se interfieren porque la a nunca puede formar parte del bb final: toda cadena del lenguaje se descompone como $u \cdot bb$, donde u contiene al menos una a, y recíprocamente cualquier $u \cdot bb$ con a en u cumple ambas condiciones.

c) Casos de prueba

Tabla 5: Casos de prueba para $(a+b)^*a(a+b)^*bb$

Cadena	Resultado esperado	AFN	AFD
abb	Aceptada	Aceptada	Aceptada
aabb	Aceptada	Aceptada	Aceptada
babbb	Aceptada	Aceptada	Aceptada
ababbb	Aceptada	Aceptada	Aceptada
babb	Aceptada	Aceptada	Aceptada
bb	Rechazada	Rechazada	Rechazada
aaa	Rechazada	Rechazada	Rechazada
aba	Rechazada	Rechazada	Rechazada
baba	Rechazada	Rechazada	Rechazada
abab	Rechazada	Rechazada	Rechazada
bab	Rechazada	Rechazada	Rechazada
ab	Rechazada	Rechazada	Rechazada

Se probaron las 4 aceptadas y 4 rechazadas indicadas en el enunciado, además de la aceptada **babb** y los casos límite **bab** (termina en un solo b, frontera inmediata con `babb`) y **ab** (tiene a y termina en b, pero carece del segundo b). Los resultados coinciden entre esperado, AFN y AFD.

d) Evidencias

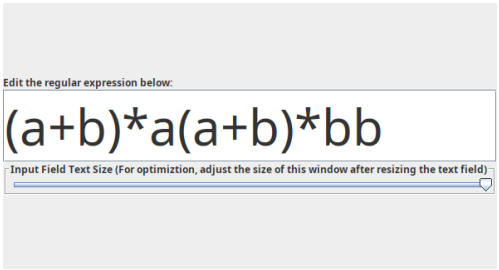


Figura 5: Expresión regular ingresada en JFLAP

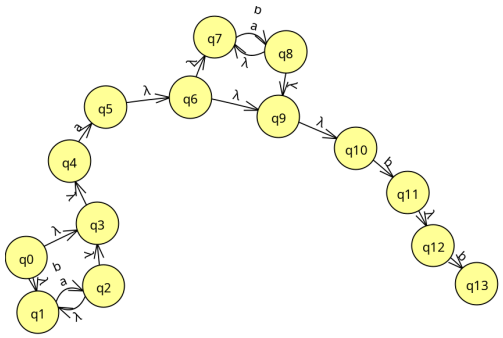


Figura 6: AFN generado (`P4_AFN.jff`)

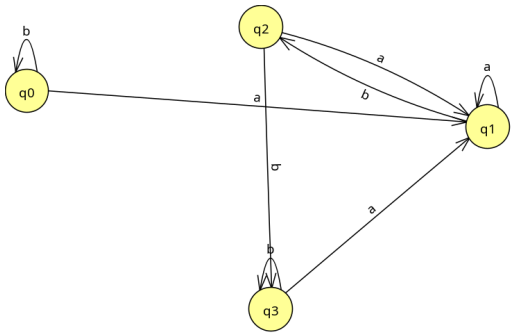


Figura 7: AFD equivalente (`P4_AFD.jff`)

e) Reflexión

Expresión Regular, AFN y AFD **no poseen distinta capacidad de reconocimiento**: son tres formas de representar la misma clase de lenguajes, los lenguajes regulares. El teorema de Kleene garantiza que un lenguaje es regular si y solo si existe una ER, un AFN y un AFD que lo reconocen. En la práctica difieren en su uso, no en su poder: la ER es una notación declarativa y compacta; el AFN, al admitir transiciones λ y no determinismo, suele construirse con menos estados; y el AFD elimina la ambigüedad, de modo que toda cadena tiene un único recorrido, lo que simplifica su implementación mecánica. En este laboratorio, las tres representaciones aceptaron y rechazaron exactamente las mismas cadenas en todas las pruebas, confirmando que solo cambia la forma de describir el mismo lenguaje.

3. Conclusiones

El laboratorio permitió relacionar de forma práctica las tres representaciones de un lenguaje regular: la expresión regular como notación declarativa, el AFN como resultado directo de la conversión de la expresión y el AFD como versión determinista equivalente. Se observó que la conversión introdujo transiciones λ y no determinismo en los AFN, mientras que los AFD resultantes eliminaron la ambigüedad conservando exactamente el lenguaje, lo que confirmó, junto con las tablas de pruebas, la equivalencia entre ER, AFN y AFD.

JFLAP demostró ser una herramienta útil para diseñar, convertir y validar autómatas: permitió verificar rápidamente la aceptación o rechazo de cada cadena y comprobar que las predicciones teóricas coincidían con los resultados simulados. Además, se reinforced la comprensión de cómo traducir restricciones sobre cadenas (inicio, final, presencia de subcadenas) a expresiones regulares correctas y a autómatas equivalentes.

4. Archivos adjuntos

Archivos `.jff` entregados junto con el informe:

- `P1_AFN.jff` – AFN de $(0+1)^*01$ (Pregunta 1)
- `P2_AFN.jff` – AFN de $a(a+b)^*b$ (Pregunta 2)
- `P3_AFN.jff` – AFN de $(0+1)^*101(0+1)^*$ (Pregunta 3)
- `P3_AFD.jff` – AFD equivalente del AFN de la Pregunta 3
- `P4_AFN.jff` – AFN de $(a+b)^*a(a+b)^*bb$ (Pregunta 4)
- `P4_AFD.jff` – AFD equivalente del AFN de la Pregunta 4

Enlace al repositorio (GitHub):
https://github.com/franciscobenavides876/Teoria-Laboratorios/tree/Laboratorio-2/Lab-2